

REDUX TOOLKIT

Cours Theorique

Session 15 : Redux Toolkit & Immer
Simplifier Redux avec les outils modernes

Module M203 - Developpement Front-End

ISTA - 2eme Annee DEVOWFS

Annee 2024/2025

Objectifs de la seance (2.5h) :

- Comprendre **pourquoi** Redux Toolkit existe
- Maitriser **configureStore** pour configurer le store
- Maitriser **createSlice** pour creer reducers + actions
- Comprendre **Immer** et la syntaxe "mutative"
- Convertir du code Redux classique vers RTK

Duree : 2h30

Prerequis : Session 14 (Redux classique)

Prochaine session : S16 - Selectors avances

Table des matières

1	Pourquoi Redux Toolkit ?	2
1.1	Les problemes de Redux classique	2
1.2	Exemple : Redux classique vs Redux Toolkit	2
1.3	Qu'est-ce que Redux Toolkit ?	3
1.4	Installation	3
2	configureStore	4
2.1	Definition	4
2.2	Comparaison : createStore vs configureStore	4
2.3	Syntaxe de configureStore	5
3	createSlice	5
3.1	Definition	5
3.2	Syntaxe de createSlice	6
3.3	Ce que createSlice genere	6
4	Immer : L'Immutabilite Simplifiee	8
4.1	Le probleme de l'immutabilite	8
4.2	La solution : Immer	8
4.3	Comparaison : Sans Immer vs Avec Immer	9
4.4	Methodes JavaScript autorisees avec Immer	10
4.5	Deux styles possibles dans createSlice	10
5	Exemple Complet : Todo App avec RTK	10
5.1	Structure du projet	10
5.2	todosSlice.js	11
5.3	store.js	12
5.4	main.jsx	12
5.5	TodoList.jsx	12
6	Recapitulatif	13
6.1	Tableau comparatif complet	13
6.2	Checklist Session 15	13

1 Pourquoi Redux Toolkit ?

1.1 Les problemes de Redux classique

En Session 14, nous avons vu Redux classique. Bien que puissant, il presente plusieurs inconvenients :

Attention

Problemes de Redux classique :

1. **Trop de code boilerplate** : Actions, types, reducers dans des fichiers separes
2. **Immutabilite difficile** : Syntaxe verbose avec spread operator (`...state`)
3. **Pas de standard** : Chaque projet organise le code differemment

1.2 Exemple : Redux classique vs Redux Toolkit

Comparaison : Redux Classique vs Redux Toolkit

Redux Classique - 3 fichiers, 40 lignes :

```
// actions/counterActions.js
export const INCREMENT = 'INCREMENT';
export const increment = () => ({ type: INCREMENT });

// reducers/counterReducer.js
const counterReducer = (state = { value: 0 }, action) => {
  switch (action.type) {
    case 'INCREMENT':
      return { ...state, value: state.value + 1 };
    default:
      return state;
  }
};

// store.js
import { createStore } from 'redux';
const store = createStore(counterReducer);
```

Redux Toolkit - 1 fichier, 15 lignes :

```
// counterSlice.js
import { createSlice, configureStore } from '@reduxjs/toolkit';

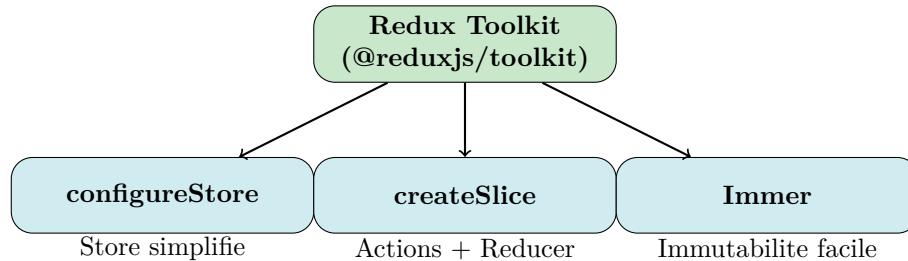
const counterSlice = createSlice({
  name: 'counter',
  initialState: { value: 0 },
  reducers: {
    increment: (state) => { state.value += 1; } // Immer !
  }
});

export const { increment } = counterSlice.actions;
const store = configureStore({ reducer: counterSlice.reducer });
```

1.3 Qu'est-ce que Redux Toolkit ?

Definition

Redux Toolkit (RTK) est la methode **officielle et recommandee** pour ecrire du code Redux. C'est un package qui inclut des utilitaires pour simplifier les taches Redux courantes.



1.4 Installation

```
# Installer Redux Toolkit et React-Redux
npm install @reduxjs/toolkit react-redux
```

Important - A retenir pour l'examen

Note : Avec Redux Toolkit, on n'installe plus `redux` separement. RTK l'inclut automatiquement.

```
# Redux Classique (S14)
npm install redux react-redux

# Redux Toolkit (S15+)
npm install @reduxjs/toolkit react-redux
```

2 configureStore

2.1 Definition

Definition

`configureStore` est une fonction de RTK qui **simplifie la creation du store**. Elle configure automatiquement :

- Les Redux DevTools
- Le middleware `redux-thunk` (pour les actions asynchrones)
- Les verifications de developpement (erreurs de mutation)

2.2 Comparaison : `createStore` vs `configureStore`

Comparaison : Redux Classique vs Redux Toolkit

Redux Classique avec `createStore` :

```
import { createStore, combineReducers, applyMiddleware } from 'redux';
import thunk from 'redux-thunk';

const rootReducer = combineReducers({
  counter: counterReducer,
  user: userReducer
});

const store = createStore(
  rootReducer,
  applyMiddleware(thunk),
  window.__REDUX_DEVTOOLS_EXTENSION__ && window.
    __REDUX_DEVTOOLS_EXTENSION__()
);
```

Redux Toolkit avec `configureStore` :

```
import { configureStore } from '@reduxjs/toolkit';

const store = configureStore({
  reducer: {
    counter: counterReducer,
    user: userReducer
  }
  // DevTools et thunk sont actives automatiquement !
});
```

2.3 Syntaxe de configureStore

Syntaxe RTK

```
1 import { configureStore } from '@reduxjs/toolkit';
2 import counterReducer from '../features/counter/counterSlice';
3 import userReducer from '../features/user/userSlice';
4
5 const store = configureStore({
6   reducer: {
7     // Chaque cle devient le chemin dans le state
8     counter: counterReducer, // state.counter
9     user: userReducer        // state.user
10  }
11 });
12
13 export default store;
```

Astuce

Structure recommandee :

```
src/
  app/
    store.js          # configureStore
  features/
    counter/
      counterSlice.js # createSlice pour counter
    user/
      userSlice.js    # createSlice pour user
    ...
```

3 createSlice

3.1 Definition

Definition

`createSlice` est une fonction de RTK qui **genere automatiquement** :

- Les **action creators** (plus besoin de les ecrire!)
- Les **types d'actions** (generes automatiquement)
- Le **reducer** complet

Tout cela dans un **seul fichier** !

3.2 Syntaxe de createSlice

Syntaxe RTK

```

1 import { createSlice } from '@reduxjs/toolkit';
2
3 const counterSlice = createSlice({
4   // Nom du slice (utilise pour generer les types d'actions)
5   name: 'counter',
6
7   // State initial
8   initialState: {
9     value: 0
10  },
11
12  // Reducers : chaque fonction devient une action
13  reducers: {
14    increment: (state) => {
15      state.value += 1; // Syntaxe "mutative" grace a Immer
16    },
17    decrement: (state) => {
18      state.value -= 1;
19    },
20    incrementByAmount: (state, action) => {
21      state.value += action.payload;
22    },
23    reset: (state) => {
24      state.value = 0;
25    }
26  }
27 });
28
29 // Exporter les actions (generees automatiquement)
30 export const { increment, decrement, incrementByAmount, reset } =
31   counterSlice.actions;
32
33 // Exporter le reducer
34 export default counterSlice.reducer;

```

3.3 Ce que createSlice genere

Vous ecrivez	RTK genere
name: 'counter'	Prefixe pour les types d'actions
reducers: { increment: ... }	Action creator increment()
	Type d'action 'counter/increment'
	Reducer complet avec switch/case

Astuce

Quand vous appelez `increment()`, RTK genere automatiquement :

```
{ type: 'counter/increment' }
```

Quand vous appelez `incrementByAmount(5)`, RTK genere :

```
{ type: 'counter/incrementByAmount', payload: 5 }
```


4 Immer : L'Immutabilite Simplifiee

4.1 Le probleme de l'immutabilite

En Redux classique, on doit **toujours retourner un nouvel objet** sans modifier l'original :

```
1 // Redux Classique : syntaxe complexe pour l'immutabilite
2 case 'ADD_ITEM':
3   return {
4     ...state,
5     items: [...state.items, action.payload]
6   };
7
8 case 'UPDATE_USER':
9   return {
10    ...state,
11    user: {
12      ...state.user,
13      address: {
14        ...state.user.address,
15        city: action.payload
16      }
17    }
18  };
```

Attention

Plus l'objet est **imbrique** (nested), plus la syntaxe devient **complexe et sujette aux erreurs** !

4.2 La solution : Immer

Definition

Immer est une bibliotheque integree dans Redux Toolkit qui permet d'ecrire du code qui **semble mutatif** mais qui produit en realite un **nouvel objet immutable**.

Important - A retenir pour l'examen

Regle fondamentale : La syntaxe "mutative" ne fonctionne **QUE** dans `createSlice` et `createReducer` de RTK. En dehors de ces fonctions, les mutations directes sont interdites !

4.3 Comparaison : Sans Immer vs Avec Immer

Comparaison : Redux Classique vs Redux Toolkit

SANS Immer (Redux Classique) :

```
// Ajouter un item
case 'ADD_ITEM':
  return {
    ...state,
    items: [...state.items, action.payload]
  };

// Supprimer un item
case 'REMOVE_ITEM':
  return {
    ...state,
    items: state.items.filter(item => item.id !== action.payload)
  };

// Modifier une propriete imbriquee
case 'UPDATE_CITY':
  return {
    ...state,
    user: {
      ...state.user,
      address: {
        ...state.user.address,
        city: action.payload
      }
    }
  }
};
```

AVEC Immer (Redux Toolkit) :

```
// Ajouter un item
addItem: (state, action) => {
  state.items.push(action.payload); // push() autorise !
},

// Supprimer un item
removeItem: (state, action) => {
  const index = state.items.findIndex(item => item.id === action.payload);
  state.items.splice(index, 1); // splice() autorise !
},

// Modifier une propriete imbriquee
updateCity: (state, action) => {
  state.user.address.city = action.payload; // Affectation directe !
}
```

4.4 Methodes JavaScript autorisees avec Immer

Operation	Classique (interdit)	Avec Immer (autorise)
Ajouter	[...arr, item]	arr.push(item)
Supprimer	arr.filter(...)	arr.splice(index, 1)
Modifier	{...obj, key: val}	obj.key = val
Vider	return []	arr.length = 0

4.5 Deux styles possibles dans createSlice

Astuce

Dans `createSlice`, vous pouvez utiliser les deux styles :

```
reducers: {  
  // Style 1 : Mutation avec Immer (recommande pour simplicite)  
  increment: (state) => {  
    state.value += 1;  
  },  
  
  // Style 2 : Retour immutable classique (toujours valide)  
  reset: (state) => {  
    return { ...state, value: 0 };  
  }  
}
```

5 Exemple Complet : Todo App avec RTK

5.1 Structure du projet

```
src/  
  app/  
    store.js  
  features/  
    todos/  
      todosSlice.js  
  components/  
    TodoList.jsx  
    TodoForm.jsx  
  App.jsx  
  main.jsx
```

5.2 todosSlice.js

```
1  import { createSlice } from '@reduxjs/toolkit';
2
3  const todosSlice = createSlice({
4    name: 'todos',
5    initialState: {
6      items: [],
7      filter: 'all' // 'all', 'active', 'completed'
8    },
9    reducers: {
10     // Ajouter une tache
11     addTodo: (state, action) => {
12       state.items.push({
13         id: Date.now(),
14         text: action.payload,
15         completed: false
16       });
17     },
18
19     // Basculer le statut completed
20     toggleTodo: (state, action) => {
21       const todo = state.items.find(t => t.id === action.payload);
22       if (todo) {
23         todo.completed = !todo.completed;
24       }
25     },
26
27     // Supprimer une tache
28     deleteTodo: (state, action) => {
29       const index = state.items.findIndex(t => t.id === action.payload);
30       if (index !== -1) {
31         state.items.splice(index, 1);
32       }
33     },
34
35     // Changer le filtre
36     setFilter: (state, action) => {
37       state.filter = action.payload;
38     }
39   }
40 });
41
42 // Exporter les actions
43 export const { addTodo, toggleTodo, deleteTodo, setFilter } = todosSlice.actions;
44
45 // Exporter le reducer
46 export default todosSlice.reducer;
```

5.3 store.js

```
1 import { configureStore } from '@reduxjs/toolkit';
2 import todosReducer from '../features/todos/todosSlice';
3
4 const store = configureStore({
5   reducer: {
6     todos: todosReducer
7   }
8 });
9
10 export default store;
```

5.4 main.jsx

```
1 import React from 'react';
2 import ReactDOM from 'react-dom/client';
3 import { Provider } from 'react-redux';
4 import store from './app/store';
5 import App from './App';
6
7 ReactDOM.createRoot(document.getElementById('root')).render(
8   <Provider store={store}>
9     <App />
10   </Provider>
11 );
```

5.5 TodoList.jsx

```
1 import { useSelector, useDispatch } from 'react-redux';
2 import { toggleTodo, deleteTodo } from '../features/todos/todosSlice';
3
4 function TodoList() {
5   const todos = useSelector((state) => state.todos.items);
6   const dispatch = useDispatch();
7
8   return (
9     <ul>
10       {todos.map((todo) => (
11         <li key={todo.id}>
12           <span
13             style={{ textDecoration: todo.completed ? 'line-through' : 'none' }}
14             onClick={() => dispatch(toggleTodo(todo.id))}
15           >
16             {todo.text}
17           </span>
18           <button onClick={() => dispatch(deleteTodo(todo.id))}>X</button>
19         </li>
20       ))}
21     </ul>
22   );
23 }
```

6 Recapitulatif

6.1 Tableau comparatif complet

Aspect	Redux Classique	Redux Toolkit
Installation	<code>redux react-redux</code>	<code>@reduxjs/toolkit react-redux</code>
Creer le store	<code>createStore()</code>	<code>configureStore()</code>
Actions	Fichier separe, ecrire manuellement	Generees par <code>createSlice</code>
Types d'actions	Constantes manuelles	Generees automatiquement
Reducer	switch/case, spread operator	Fonctions dans <code>reducers: {}</code>
Immutabilite	Manuelle (<code>...state</code>)	Automatique avec Immer
DevTools	Configuration manuelle	Automatique
Middleware	<code>applyMiddleware()</code>	Inclus par default

6.2 Checklist Session 15

- Comprendre les **limites de Redux classique**
- Installer `@reduxjs/toolkit`
- Utiliser `configureStore` pour creer le store
- Utiliser `createSlice` pour creer actions + reducer
- Comprendre **Immer** et la syntaxe "mutative"
- Exporter `slice.actions` et `slice.reducer`

Prochaine session (S16) : Selectors avances

Nous verrons `createSelector` pour optimiser les performances et creer des selectors memoises.